Lab 2

This lab includes:

- Mathematical formulation
- Activation functions with derivatives
- Gradient-based learning
- Adaline And Perceptron implementation from scratch
- Logical gates (AND, OR, XOR)
- Decision boundary visualization
- Real life data and the difference between Adaline and Perceptron training

```
import numpy as np
import matplotlib.pyplot as plt
```

1- Mathematical Foundation

Linear Model: [z = w^T x + b]

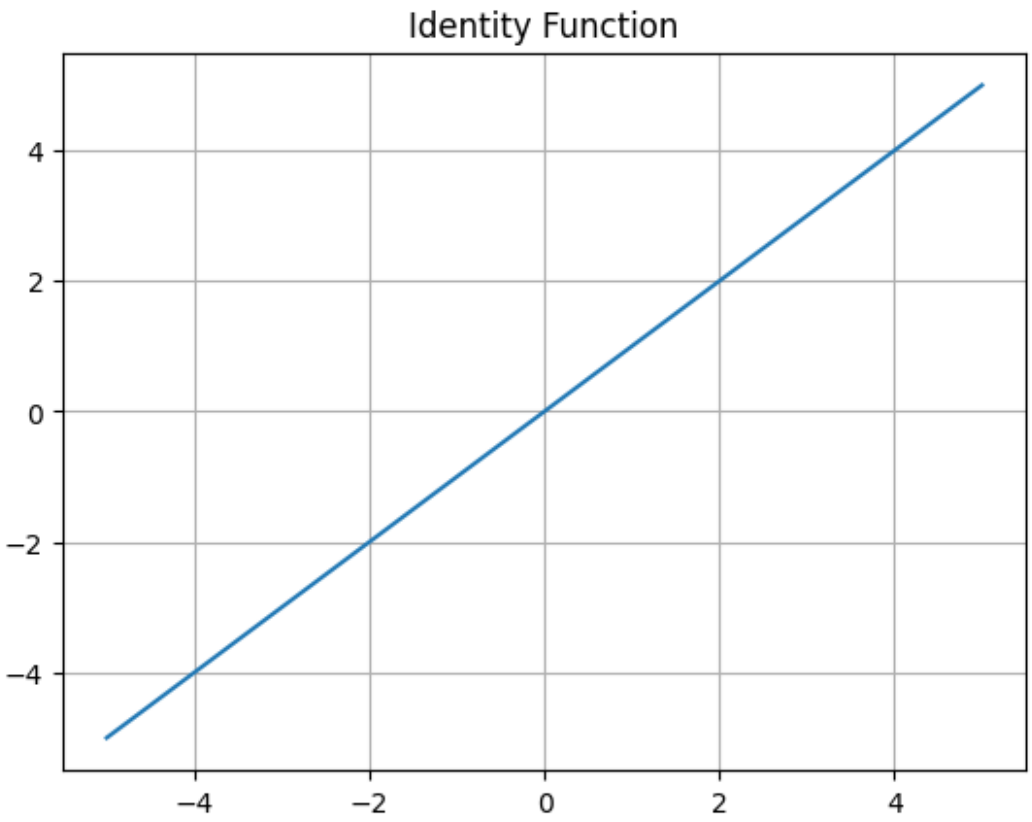
Activation: [y = \sigma(z)]

Loss Function (MSE): [MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2]

2- Activation Functions

Identity

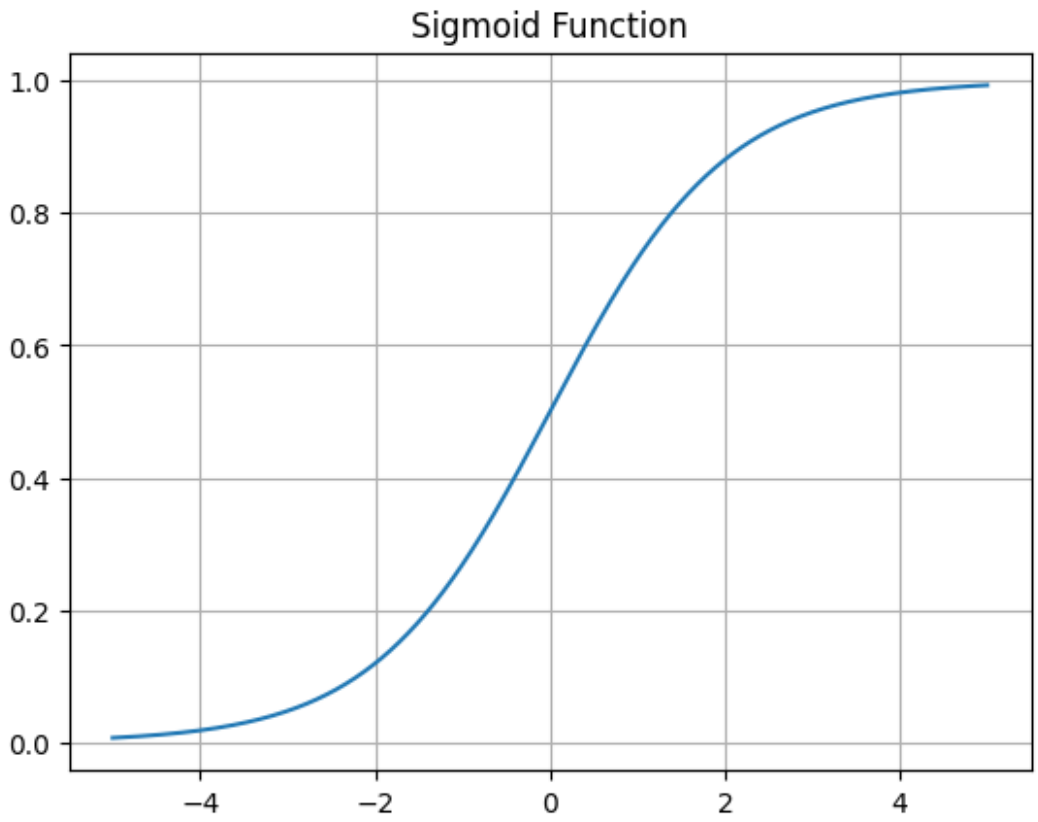
```
def identity(x):
    return x

x = np.linspace(-5,5,1000)
plt.figure()
plt.plot(x, identity(x))
plt.title('Identity Function')
plt.grid()
plt.show()
```



Sigmoid

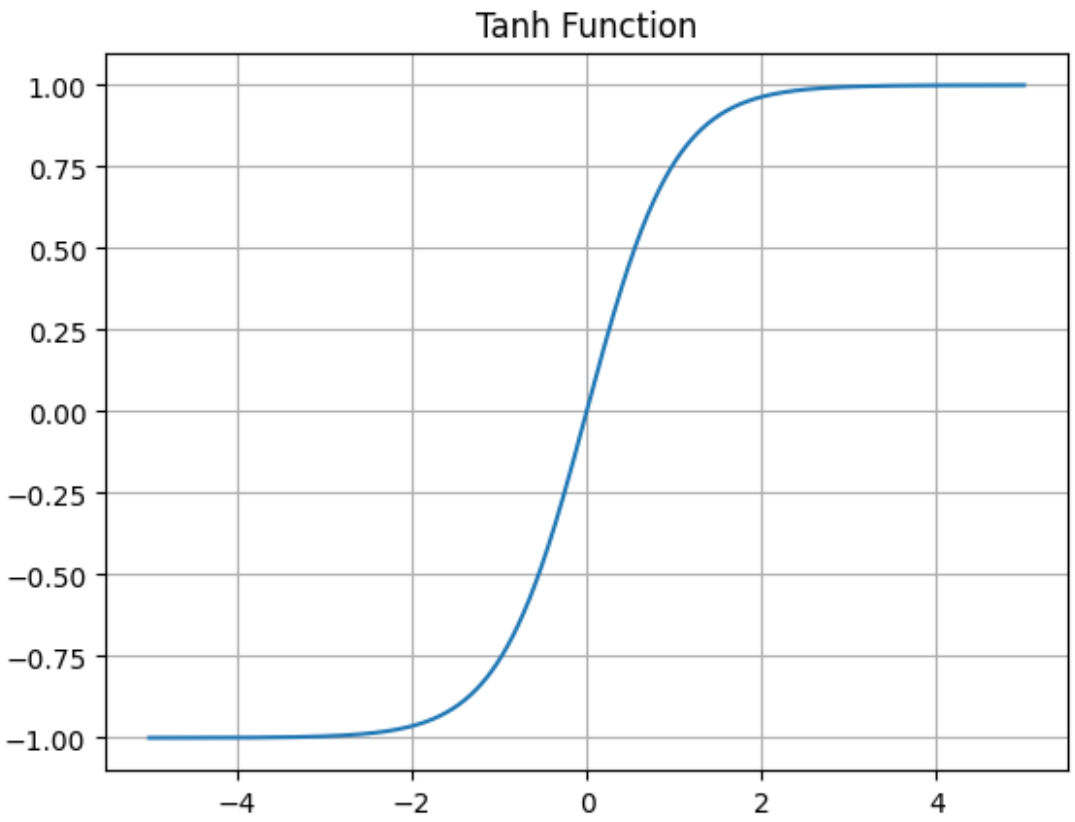
[\sigma(x)=\frac{1}{1+e^{-x}}]

```
def sigmoid(x):
    return 1/(1+np.exp(-x))

def sigmoid_derivative(x):
    s = sigmoid(x)
    return s*(1-s)

plt.figure()
plt.plot(x, sigmoid(x))
plt.title('Sigmoid Function')
plt.grid()
plt.show()
```



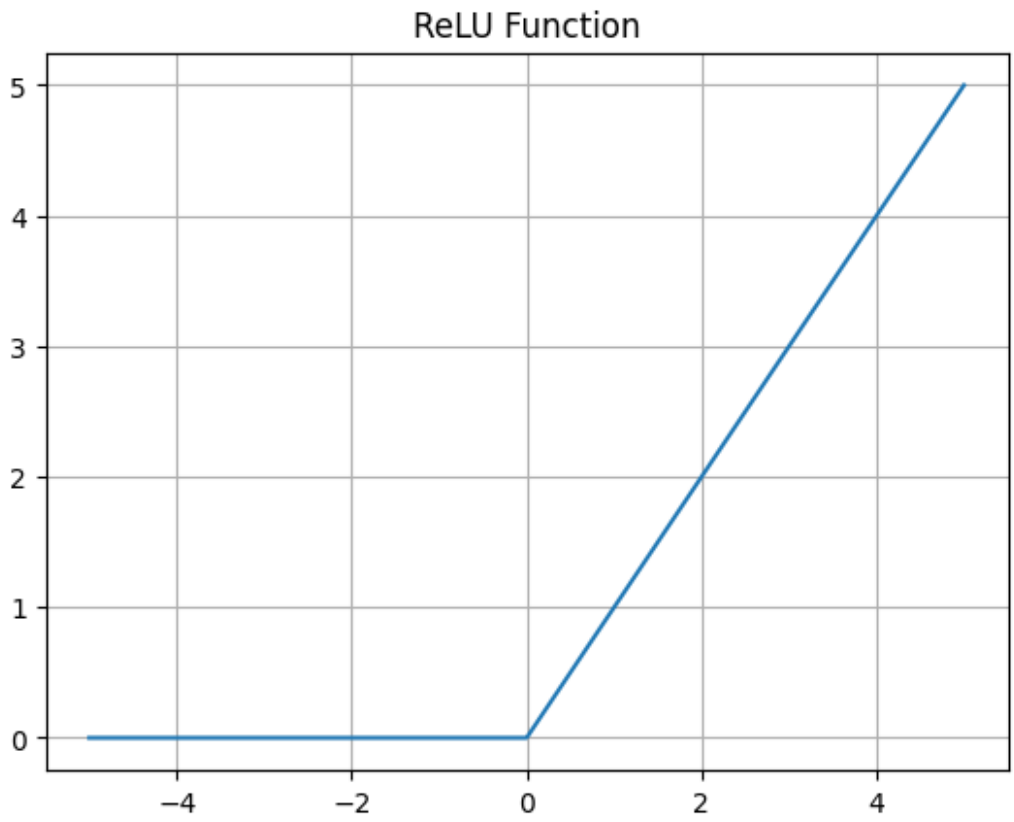
Tanh

```
def tanh(x):
    return np.tanh(x)

plt.figure()
plt.plot(x, tanh(x))
plt.title('Tanh Function')
plt.grid()
plt.show()
```



ReLU

```
def relu(x):
    return np.maximum(0,x)

plt.figure()
plt.plot(x, relu(x))
plt.title('ReLU Function')
plt.grid()
plt.show()
```



3- Perceptron and Adaline Implementation From Scratch

```
class Perceptron:

    def __init__(self, learning_rate=0.1, epochs=50):
        self.lr = learning_rate
        self.epochs = epochs

    def fit(self, X, y):
        n_samples, n_features = X.shape

        self.w = np.zeros(n_features)
        self.b = 0
        self.converged = False

        for _ in range(self.epochs):
            errors = 0
            for i in range(n_samples):

                z = np.dot(X[i], self.w) + self.b
                y_pred = 1 if z >= 0 else -1

                if y[i] != y_pred:
                    self.w += self.lr * y[i] * X[i]
```

```
                    self.b += self.lr * y[i]
                    errors += 1

            if errors == 0:
                self.converged = True
                break

        print(f"Perceptron converged: {self.converged}")

    def predict(self, X):
        z = np.dot(X, self.w) + self.b
        return np.where(z >= 0, 1, -1)

class Adaline:

    def __init__(self, learning_rate=0.01, epochs=100, tol=1e-2):
        self.lr = learning_rate
        self.epochs = epochs
        self.tol = tol

    def fit(self, X, y):
        n_samples, n_features = X.shape

        self.w = np.zeros(n_features)
        self.b = 0
        self.losses = []
        self.converged = False

        for _ in range(self.epochs):

            z = np.dot(X, self.w) + self.b
            errors = y - z

            # This is technically a shorthand version of gradient
descent
            self.w += self.lr * np.dot(X.T, errors)
            self.b += self.lr * np.sum(errors)

            loss = np.mean(errors**2)
            self.losses.append(loss)

            if len(self.losses) > 1 and abs(self.losses[-2] -
self.losses[-1]) < self.tol:
                self.converged = True
                break

        print(f"Adaline converged: {self.converged}")

    def predict(self, X):
```

```
        z = np.dot(X, self.w) + self.b
        return np.where(z >= 0, 1, -1)
```

5- Logical Gates

```
# And
X = np.array([
    [ 1,  1],
    [ 1, -1],
    [-1,  1],
    [-1, -1]
])

y = np.array([1, -1, -1, -1])

perceptron = Perceptron(learning_rate=0.1, epochs=20)
perceptron.fit(X, y)

print("Perceptron Weights:", perceptron.w)
print("Perceptron Bias:", perceptron.b)

print("Predictions:", perceptron.predict(X))
```

```
Perceptron converged: True
Perceptron Weights: [0.1 0.1]
Perceptron Bias: -0.1
Predictions: [ 1 -1 -1 -1]
```

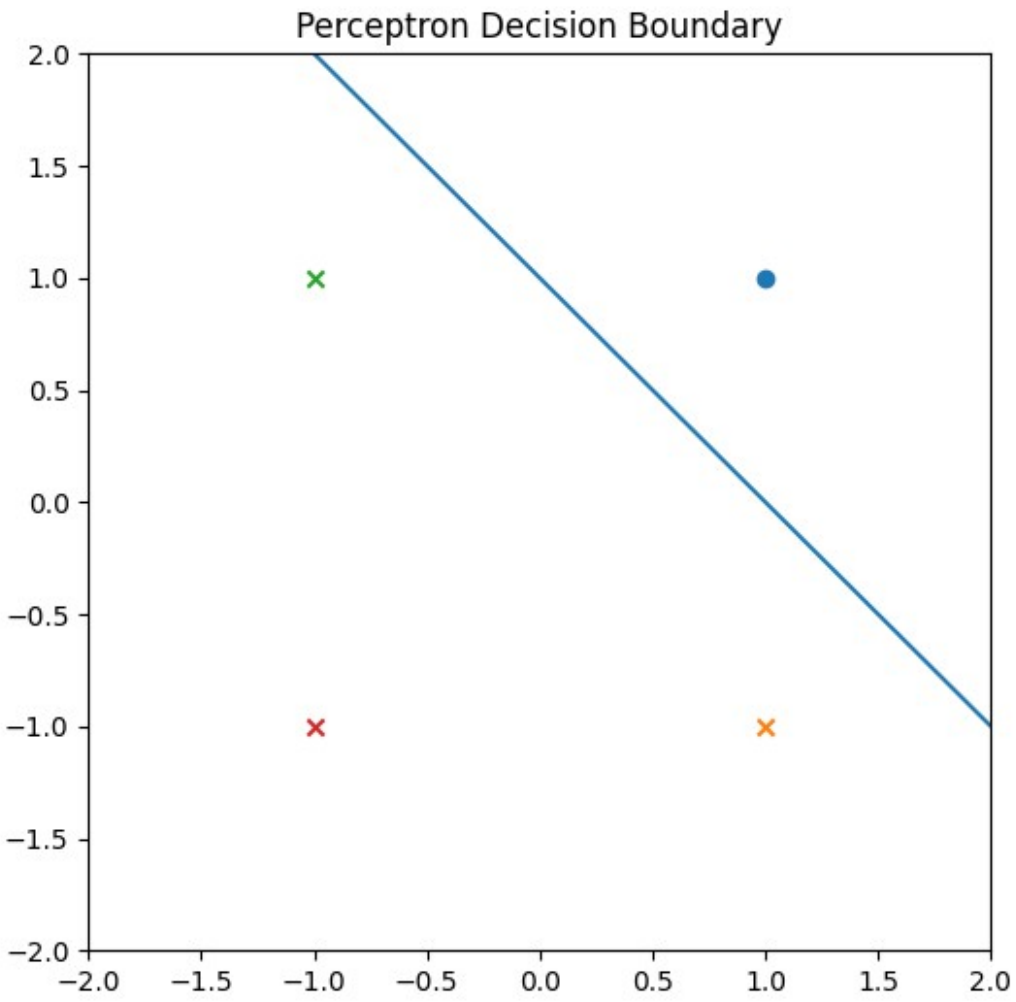
```
adaline = Adaline(learning_rate=0.01, epochs=50)
adaline.fit(X, y)

print("Adaline Weights:", adaline.w)
print("Adaline Bias:", adaline.b)

print("Predictions:", adaline.predict(X))
```

```
Adaline converged: True
Adaline Weights: [0.31229338 0.31229338]
Adaline Bias: -0.3122933766364488
Predictions: [ 1 -1 -1 -1]
```

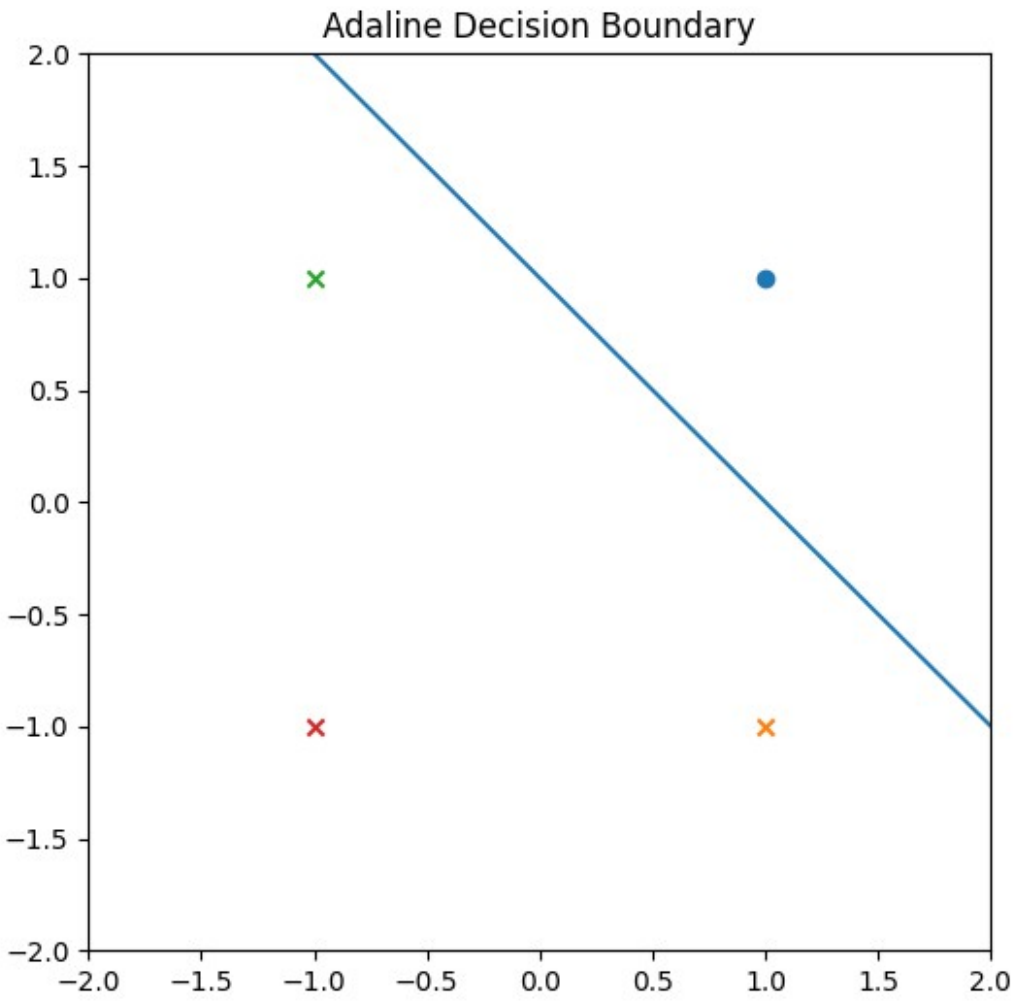
Decision Boundary Function

```
def plot_decision_boundary(model, X, y, title):

    plt.figure(figsize=(6,6))

    # Plot samples
    for i in range(len(X)):
        if y[i] == 1:
```

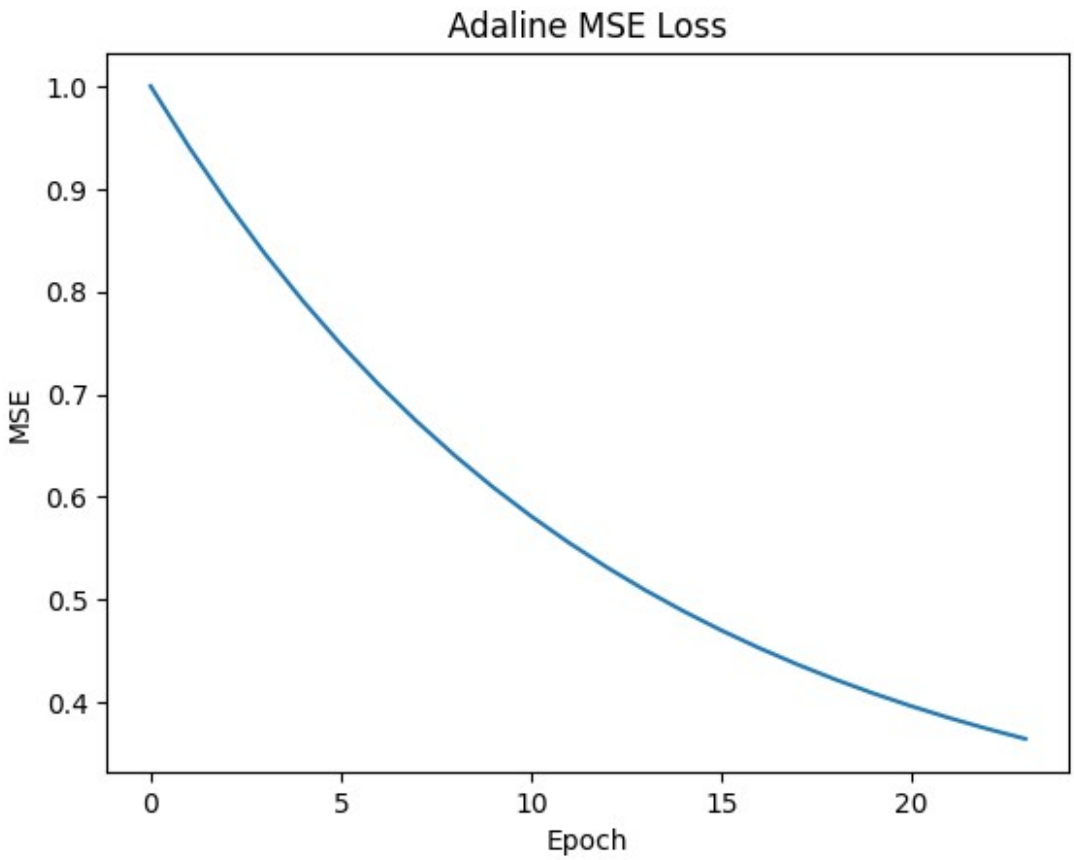
```
            plt.scatter(X[i,0], X[i,1], marker='o')
        else:
            plt.scatter(X[i,0], X[i,1], marker='x')

    w1, w2 = model.w
    b = model.b

    x_vals = np.linspace(-2, 2, 100)

    # Handle vertical line case
    if abs(w2) < 1e-10:
        # Vertical decision boundary
        x_vertical = -b / w1
        plt.axvline(x=x_vertical)
    else:
        y_vals = -(w1*x_vals + b) / w2
        plt.plot(x_vals, y_vals)

    plt.title(title)
    plt.xlim(-2,2)
    plt.ylim(-2,2)
    plt.show()

plot_decision_boundary(perceptron, X, y, "Perceptron Decision
Boundary")
```




```
plot_decision_boundary(adaline, X, y, "Adaline Decision Boundary")
```




```
plt.plot(adaline.losses)
plt.title("Adaline MSE Loss")
plt.xlabel("Epoch")
plt.ylabel("MSE")
plt.show()
```



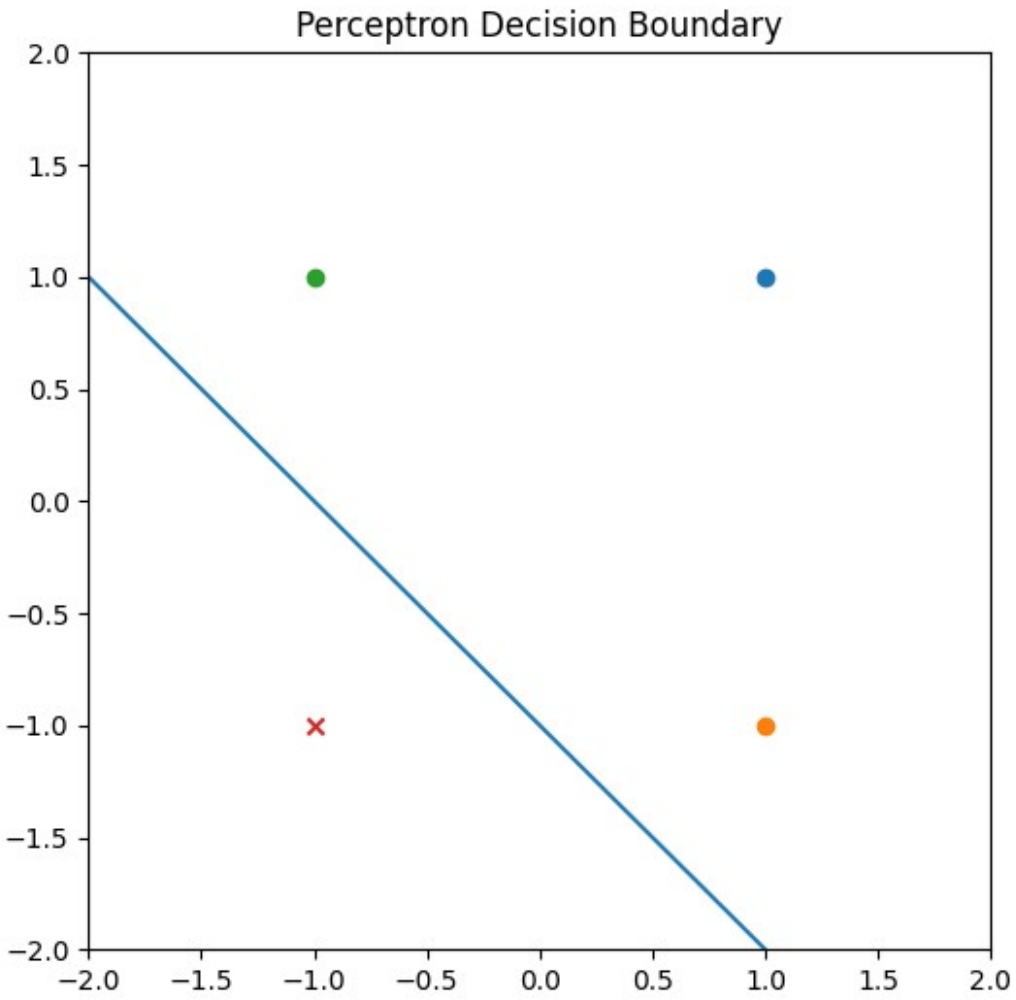

```
# OR
X = np.array([
    [ 1,  1],
    [ 1, -1],
    [-1,  1],
    [-1, -1]
])

y = np.array([1, 1, 1, -1])

perceptron = Perceptron(learning_rate=0.1, epochs=20)
perceptron.fit(X, y)

print("Perceptron Weights:", perceptron.w)
print("Perceptron Bias:", perceptron.b)

print("Predictions:", perceptron.predict(X))
```

```
Perceptron converged: True
Perceptron Weights: [0.1 0.1]
Perceptron Bias: 0.1
Predictions: [ 1  1  1 -1]
```

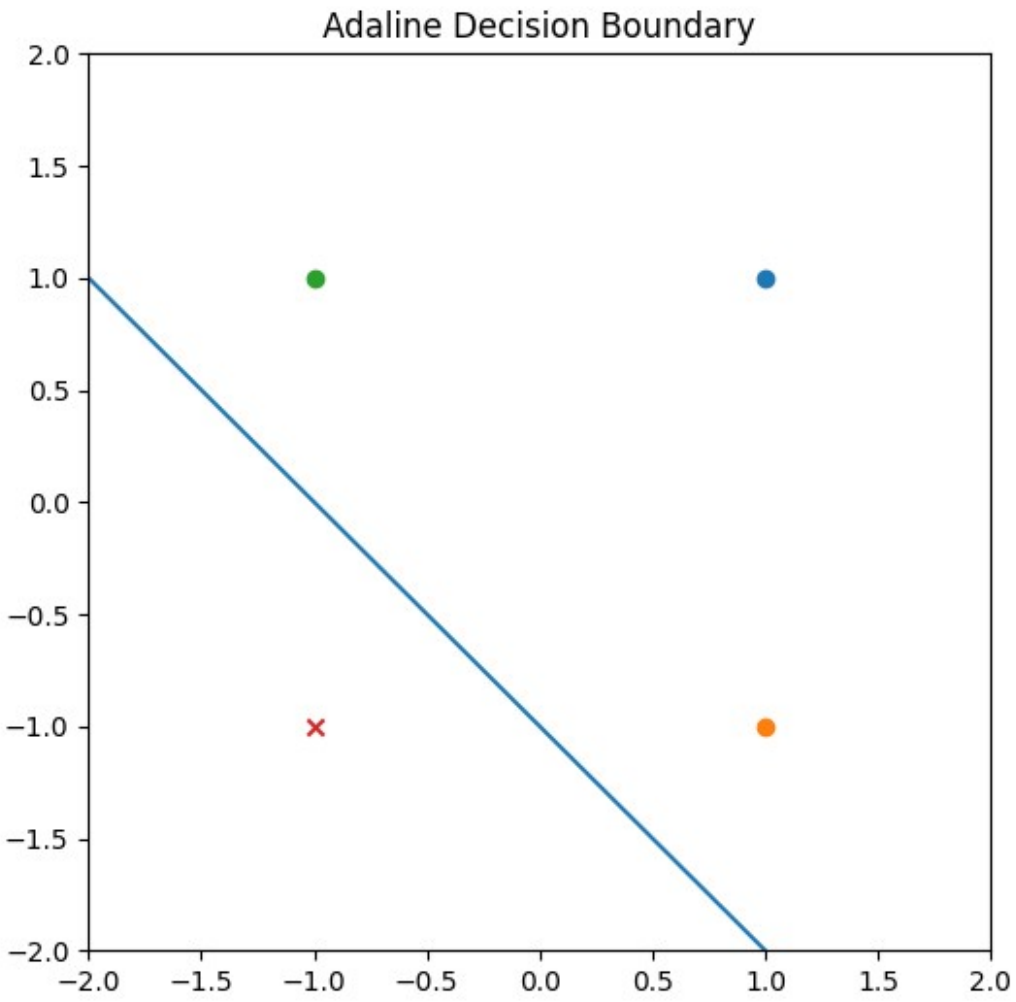
```
adaline = Adaline(learning_rate=0.01, epochs=50)
adaline.fit(X, y)

print("Adaline Weights:", adaline.w)
print("Adaline Bias:", adaline.b)

print("Predictions:", adaline.predict(X))
```

```
Adaline converged: True
Adaline Weights: [0.31229338 0.31229338]
Adaline Bias: 0.3122933766364488
Predictions: [ 1  1  1 -1]
```

```
plot_decision_boundary(perceptron, X, y, "Perceptron Decision
Boundary")
```

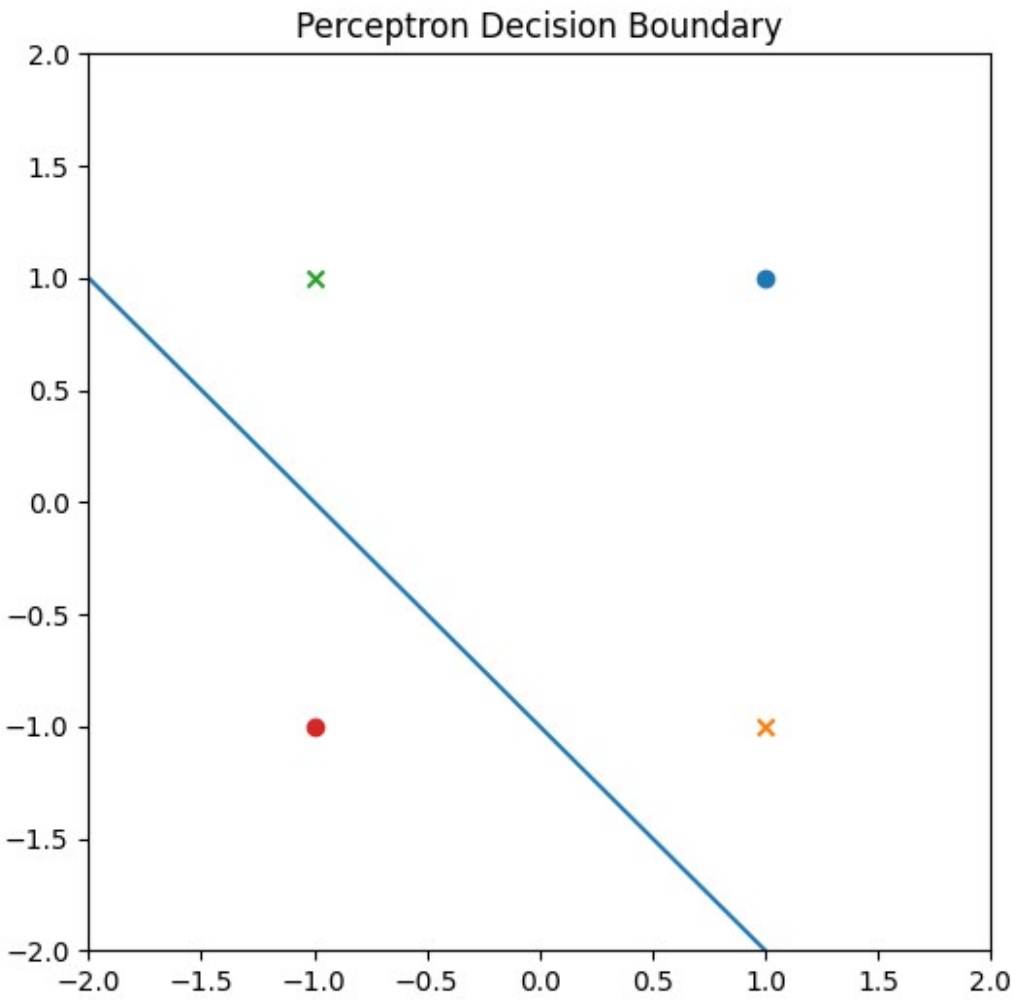



```
plot_decision_boundary(adaline, X, y, "Adaline Decision Boundary")
```



5- XOR Failure (Not Linearly Separable)

```
y_xor = np.array([1, -1, -1, 1])

perceptron_xor = Perceptron(0.1, 50)
perceptron_xor.fit(X, y_xor)

print("XOR Predictions:", perceptron_xor.predict(X))

Perceptron converged: False
XOR Predictions: [-1 -1 -1  1]

plot_decision_boundary(perceptron_xor, X, y_xor, "Perceptron Decision
Boundary")
```



6- Real life data prediction using a perceptron.

Both the Perceptron and ADALINE learn a linear decision boundary, but differ in what drives their weight updates. The Perceptron updates only on misclassifications using the thresholded binary output, so it has no explicit loss function and is only guaranteed to converge on linearly separable data. ADALINE updates on every iteration using the continuous net input before thresholding, minimizing MSE via gradient descent, which means it always converges to a stable solution regardless of linear separability.

```
import pandas as pd

df = pd.read_csv("loan_approval_data.csv")
df.head()
```

	annual_income_k	credit_score	loan_approved
0	28	580	-1
1	32	610	-1
2	25	540	-1

```
3                30            595              -1
4                22            560              -1
```

```
X = df[["annual_income_k", "credit_score"]].to_numpy()
y = df["loan_approved"].to_numpy()

X = (X - X.mean(axis=0)) / X.std(axis=0)

df["loan_approved"].value_counts()
```

```
loan_approved
-1    20
 1    20
Name: count, dtype: int64
```

```
perceptron = Perceptron(learning_rate=0.01, epochs=30)
perceptron.fit(X, y)
prediction = perceptron.predict(X)
print("Perceptron Weights:", perceptron.w)
print("Perceptron Bias:", perceptron.b)
print("Predictions:", prediction)
```

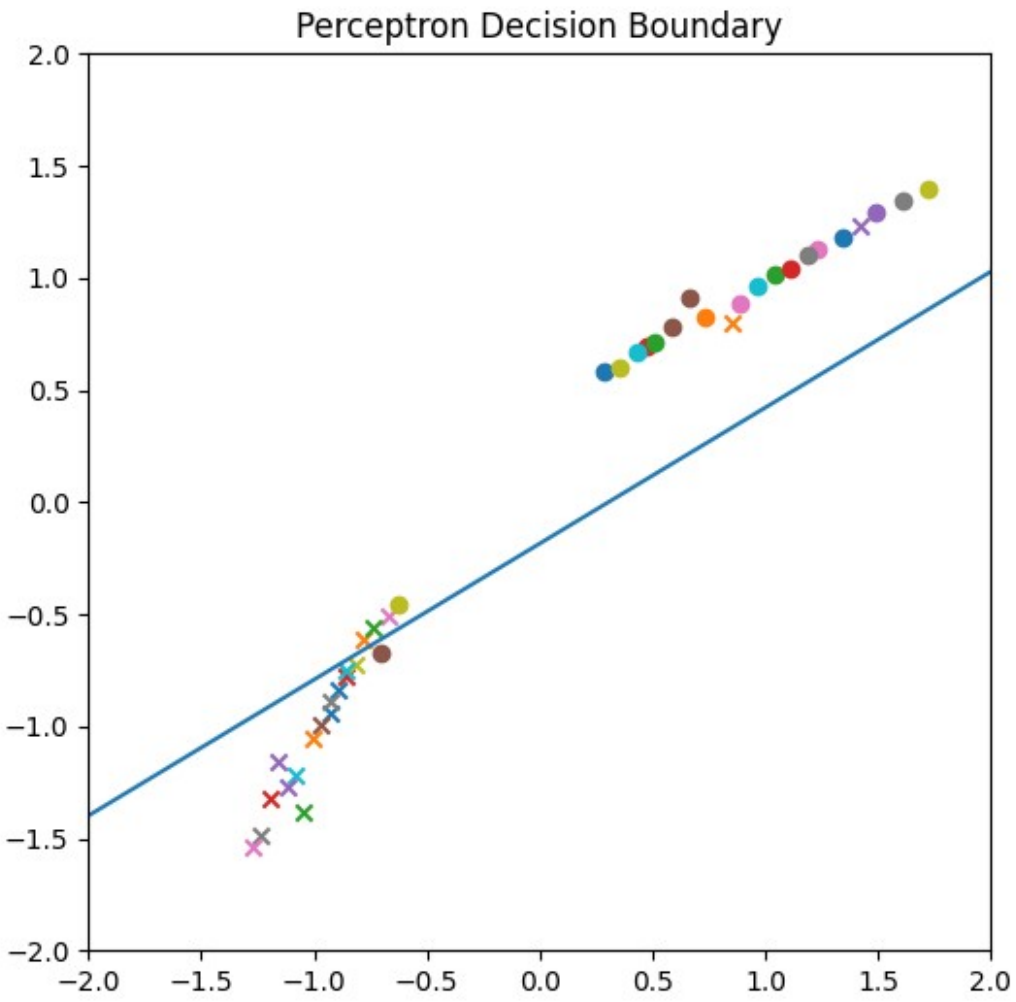
```
Perceptron converged: False
Perceptron Weights: [-0.03279746  0.054067  ]
Perceptron Bias: 0.01
Predictions: [-1  1 -1 -1 -1 -1  1 -1 -1 -1 -1 -1  1 -1 -1 -1 -1 -1  1
-1  1  1  1  1
  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1]
```

```
adaline = Adaline(learning_rate=0.01, epochs=30)
adaline.fit(X, y)
prediction = adaline.predict(X)

print("Predictions:", prediction)
```

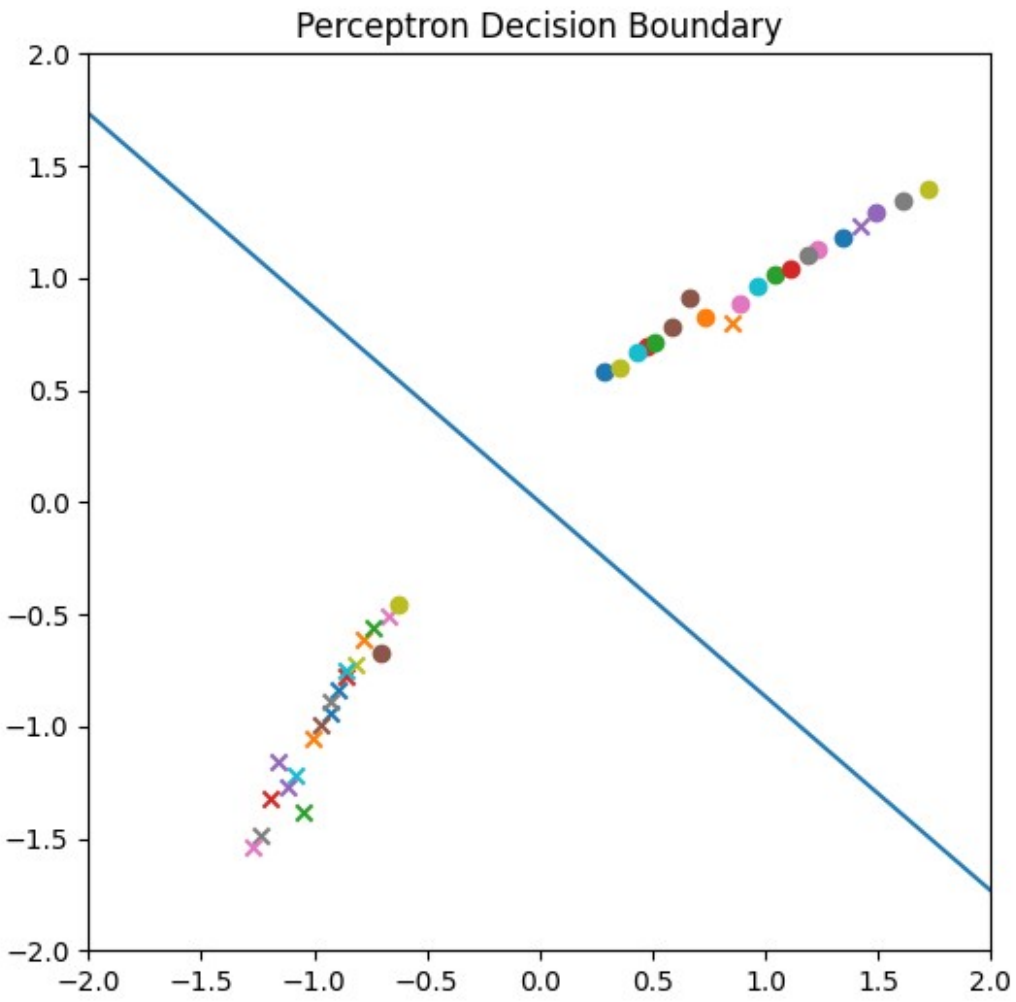
```
Adaline converged: True
Predictions: [-1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1
-1  1  1  1  1
  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1]
```

Even though predictions are close in both the Perceptron and Adaline, you can see how the linear line in the Perceptron doesn't minimize error.

```
plot_decision_boundary(perceptron, X, y, "Perceptron Decision
Boundary")
```




```
plot_decision_boundary(adaline, X, y, "Perceptron Decision Boundary")
```


Perceptron Decision Boundary
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2.0
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0